

Unitree Week 13

Presentation by Mario Ortiz

This Week

- Preparing Mimic Learning on Isaac Sim and Isaac Labs for the Unitree G1

Exploring isaacSim/IsaacLabs

After searching the IsaacLab repository to find files for imitation learning, I have found these files:

- `manager_based_rl_mimic_env.py`
 - A class file that can allow us to feed human demonstrations (such as videos) into the simulation.
- `mimic_env_cfg.py`
 - Allows us to define subtasks, breaking long actions into small steps. (Example: teach a G1 to walk then stop and pick up an item)
- `Pink_task_space_actions.py`
 - A high level inverse kinematics engine. Can be used for various tasks such as an arm controller.

Other Important Tools

- In `C:\Users\Mario\IsaacLab\scripts\imitation_learning\isaacclab_mimic`
- `annotate_demos.py`
- `generate_dataset.py`
- `.\source\isaacclab_tasks\isaacclab_tasks\manager_based\manipulation\lift\lift_env_cfg.py`
 - This provides code to train a robot to lift an object.

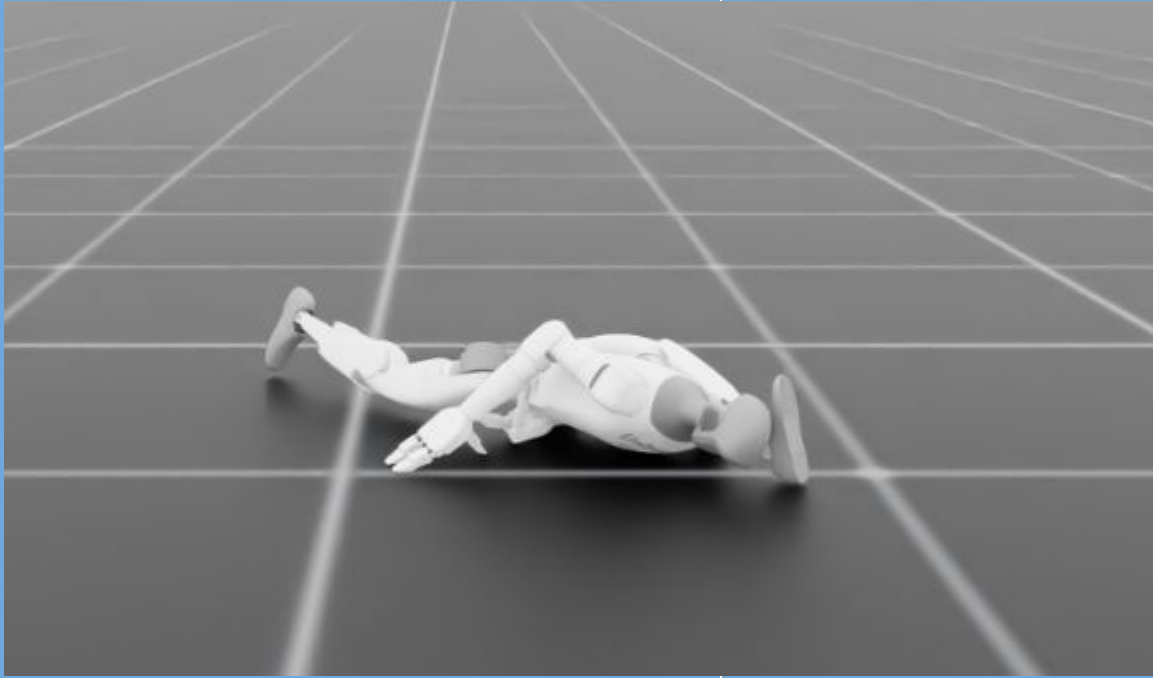
Scripted Inverse Kinematics Generator

- Scripted IK Kinematics allows us to use code to calculate and set parameters such as rotations and positions of a connected chain of joints.
- I will be creating a Scripted IK code to simulate the Unitree G1 picking up an object in the floor.

Scripted Inverse Kinematics Generator

```
C:\Users> Menu > IsaacLab > scripts > gl_manipulation > @ gl_ik_generator.py > ...
1 import argparse
2 from isaacsim.app import AppLauncher
3
4 # 1. Setup App Launcher
5 parser = argparse.ArgumentParser(description="G1 Scripted IK Generator - Verified Sync")
6 AppLauncher.add_app_launcher_args(parser)
7 args_cli = parser.parse_args()
8 app_launcher = AppLauncher(args_cli)
9 simulation_app = app_launcher.app
10
11 # 2. Imports - Synced with run_diff_ik.py
12 import torch
13 import isaacsim as sim_utils
14 from isaacsim.assets import Articulation, ArticulationCfg
15 from isaacsim.controllers import DifferentialIKController, DifferentialIKControllerCfg
16 from isaacsim.managers import SceneEntityCfg
17 from isaacsim.scene import InteractiveScene, InteractiveSceneCfg
18 from isaacsim.utils import configclass
19 from isaacsim.utils.math import subtract_frame_transforms
20
21 # Pre-defined G1 Config
22 from isaacsim.assets.robots.unireg import G1_Cfg
23
24 @configclass
25 class G1SceneCfg(InteractiveSceneCfg):
26     ground = AssetBaseCfg(prm_path="/World/defaultGroundPlane", spawn=sim_utils.GroundPlaneCfg())
27     dome_light = AssetBaseCfg(prm_path="/World/light", spawn=sim_utils.DomeLightCfg(intensity=3000.0))
28
29     # Spawn G1 with fixed base for recording
30     robot = G1_Cfg.replace(prm_path="/env/robot/Robot")
31     robot.spawn.fixed_base = True
32
33 def run_simulator(sim: sim_utils.SimulationContext, scene: InteractiveScene):
34     robot = scene["robot"]
35
36     # 3. Controller Setup - Matching Tutorial Names
37     diff_ik_cfg = DifferentialIKControllerCfg(command_type="pose", use_relative_mode=False, ik_method="dta")
38     diff_ik_controller = DifferentialIKController(diff_ik_cfg, num_envs=scene.num_envs, device=sim.device)
39
40     # Resolve end effector (Left Pail)
41     # Note: G1 joint names usually follow 'left_arm_joint_3' etc.
42     # Adjust joint_names regex if your specific URDF uses different naming.
43     robot_entity_cfg = scene.entity_cfg["robot", joint_names=["left_*"], body_names=["left_pail_link"]]
44     robot_entity_cfg.resolve(scene)
45
46     # Fixed base Jacobian index adjustment (from tutorial)
47     ee_jacobi_idx = robot_entity_cfg.body_ids[0] - 1
48
49     sim_dt = sim.get_physics_dt()
50     count = 0
51
52     while simulation_app.is_running():
53         # Obtain quantities
54         jacobian = robot.root_physx_view.get_jacobians()[[ee_jacobi_idx, robot_entity_cfg.joint_ids]]
55         ee_pose_w = robot.data.body_pose_w[:, robot_entity_cfg.body_ids[0]]
56         root_pose_w = robot.data.root_pose_w
57         joint_pos = robot.data.joint_pos[:, robot_entity_cfg.joint_ids]
58
59         # Compute frame in root frame (Crucial for accuracy)
60         ee_pos_b, ee_quat_b = subtract_frame_transforms(
61             root_pose_w[:, 0:3], root_pose_w[:, 3:7], ee_pose_w[:, 0:3], ee_pose_w[:, 3:7]
62         )
63
64         # Define a target (hovering in front of robot)
65         # In the next steps, we will make this move towards a cube!
66         ik_commands = torch.tensor([[0.4, 0.2, 0.5, 1, 0, 0, 0]], device=sim.device).repeat(scene.num_envs, 1)
67         diff_ik_controller.set_command(ik_commands)
68
69         # Compute joint commands
70         joint_pos_des = diff_ik_controller.compute(ee_pos_b, ee_quat_b, jacobian, joint_pos)
71
72         # Apply
73         robot.set_joint_position_target(joint_pos_des, joint_ids=robot_entity_cfg.joint_ids)
74         scene.write_data_to_sim()
75         sim.step()
76         count += 1
77         scene.update(sim_dt)
```

Currently Scripted IK does not work



Other methods to make HDF5

Sources

- https://isaac-sim.github.io/IsaacLab/main/source/overview/core-concepts/sensors/ray_caster.html
- <https://github.com/isaac-sim/IsaacLab/issues/4033>
- <https://www.youtube.com/watch?v=JfdPxD2vmIY>
- <https://www.youtube.com/watch?v=zIWcqZCyE-s>
-